

# 11/3 논문 - BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

https://arxiv.org/pdf/1810.04805

## 1. Introduction

### 논문이 다루는 분야

자연어 처리 중 자연어 추론 , 바꿔쓰기 판별, 개체명 인식, 질의응답

**기존 전략**
공통점: 단방향 언어 모델로 사전학습. ELMo는 정방향·역방향을 따로 학습한 뒤 연결 → BERT처럼 매 층에서 좌우 문맥을 함께 보는 구조는 아님.
특징 기반(feature-based): 사전학습 모델에서 표현을 꺼내 별도 모델의 입력으로 사용. BERT 실험에서는 BERT 고정 → 별도 과제 모델만 학습.

- 미세조정(fine-tuning): 출력층을 조금 추가 → 실제 과제에 맞게 BERT 전체와 출력층을 함께 수정.

### 해당 task에서 기존 연구 한계점

단방향 언어모델의 사용이 사전학습된 표현의 능력을 제한, fine-tuning 에서의 제약이 큼

표준 언어 모델이 단방향 → 사전학습 사용 구조 선택이 제한.

⇒ 문장 수준 과제에 최적이 아님

→ 양방향 문맥이 중요한 질의응답 등 토큰 수준 과제에서 단방향 제약이 특히 불리하다.

### 논문의 contributions

masked language model

- MLM은 입력의 일부 토큰을 예측 대상으로 선택하고, 좌우 문맥을 함께 사용해 원래 토큰을 예측하도록 한다. 이를 통해 심층 양방향 사전학습이 가능해진다.
- 다음 문장 예측 과제도 사용

→ 언어 표현에서 양방향 사전학습의 중요성 입증

→ 과제별 구조의 필요성을 줄임

## 2. Related Work

1. 비지도 특징 기반 접근 법

   a. 비신경망 방법

   b. 신경망 방법

   c. 사전학습된 단어 임베딩

      - 왼 →오로 진행하는 언어 모델링 목적 함수 & 좌우 문맥에서 올바른 단어와 잘못된 단어를 구분하는 목적함수

   d. 문장 임베딩

   e. 문단 임베딩

      - 문장 표현 학습 : 이전 문장 표현이 주어졌을 때 왼→오로 단어 생성하는 목적함수 & 잡음 제거 오토인코더에서 유도한 목적함수

   f. ELMo와 선행 모델: 정방향 언어 모델과 역방향 언어 모델을 독립적으로 학습하고, 각 방향의 표현을 연결해 문맥에 민감한 단어 표현을 얻는다.

      - 기존 과제별 구조에 문맥 표현을 추가하여 질의응답, 감성 분석, 개체명 인식 등에서 당시 최고 성능을 갱신했다.

   g. Melamud et al.(2016): LSTM으로 좌우 문맥에서 하나의 단어를 예측해 문맥 표현을 학습했다. 특징 기반 접근법이며 BERT처럼 심층 양방향 Transformer를 학습한 것은 아니다.

   h. Cloze 과제를 통해 텍스트 생성 모델의 강건성 개선

2. 비지도 미세조정 접근 법

   a. 레이블 x → 단어 임베딩 매개변수만 사전학습

   b. 문장·문서 인코더를 레이블 없는 텍스트로 사전학습한 뒤 지도학습 과제로 미세조정한다. 장점은 모델 전체의 매개변수가 적다는 것이 아니라, 처음부터 새로 학습해야 하는 매개변수가 적다는 것이다. OpenAI GPT는 GLUE의 여러 문장 수준 과제에서

당시 최고 성능을 보였다. 기존 사전학습에는 정방향 언어 모델링·오토인코더 목적함수가 사용되었다.

3. 지도학습 데이터로부터의 전이 학습
    1. 자연어 추론·기계번역처럼 대규모 지도학습 데이터가 있는 과제로 학습한 표현도 다른 과제로 전이할 수 있다.
    2. 컴퓨터 비전에서도 ImageNet 사전학습 후 미세조정하는 방식이 효과적이었다.

# 3. 제안 방법론

## Main Idea

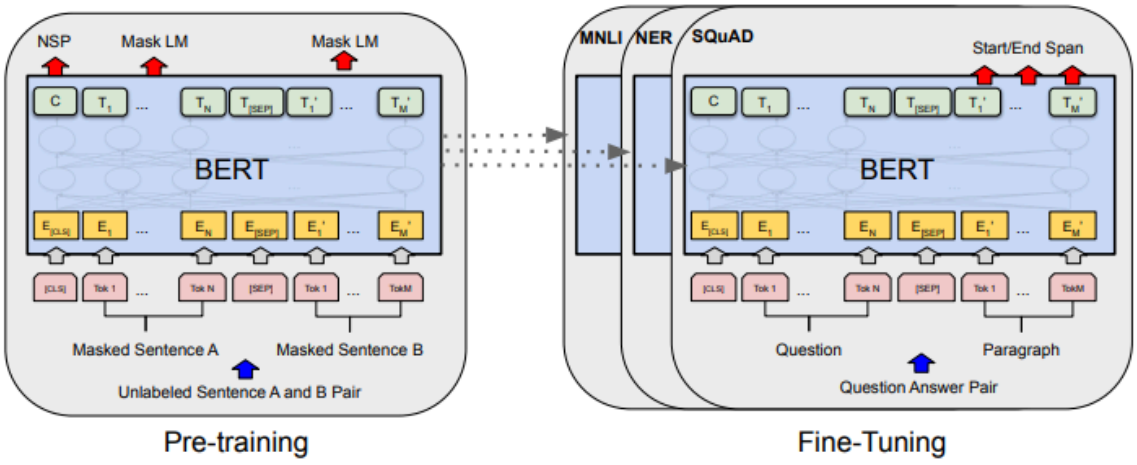
**서로 다른 과제에 걸쳐 구조가 통일됨.**

1. 사전학습

    a. :레이블이 없는 데이터를 사용, 여러 사전학습 과제에 걸쳐 모델을 학습

2. 미세조정

    a. 사전학습 가중치를 각 다운스트림 모델의 시작값으로 가져오고, 레이블이 있는 과제 데이터로 모든 매개변수를 미세조정한다. 같은 사전학습 체크포인트에서 출발하더라도 과제별로 별도의 모델을 만든다. 여러 과제를 하나의 모델에 순차적으로 계속 학습한다는 뜻은 아니다.




3. 모델 구조

    a. 다층 양방향 Transformer Encoder

b. 토큰별 벡터→ Multi-Head Self-Attention→잔차 연결 + Layer Normalization→Feed-Forward Network→잔차 연결 + Layer Normalization→문맥이 반영된 토큰별 벡터

c. Base : L = 12, H = 768, A = 12, 매개변수 1억 1000만개

비교를 위해 GPT와 동일한 모델 크기, 차이는 양방향 self attention이냐 왼쪽만 보는 self attention이냐

d. Large: L = 24, H = 1024, A = 16, 매개변수 3억 4000만개

e. self attention :  토큰이 어떤 토큰의 정보를 얼마나 가져올지

f. Multi-Head Attention: 여러 헤드가 동시에 서로 다른 관계를 살펴봄 → 결과를 합쳐 문맥 파악. 헤드를 여러 개 쓰는 것과 층을 여러 개 쌓는 것은 다름.

4. 입력/출력

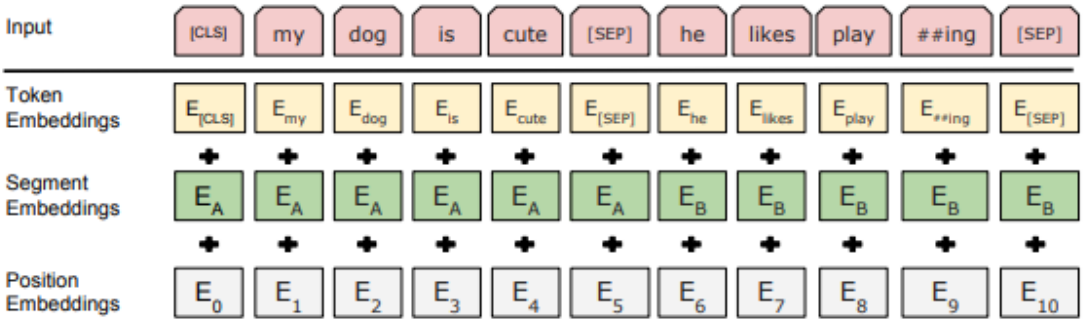
a. 입력: 단일 문장과 문장쌍 모두 →  하나의 토큰 시퀀스

- 문장: 임의의 연속된 텍스트 구간
- 시퀀스: 하나의 문장 혹은 두 문장을 함께 묶은 것

b. WordPiece 임베딩

- 첫 번째 토큰은 [CLS]. 입력은 [CLS] 문장 A [SEP] 문장 B [SEP] 형태이며, 단일 문장은 [CLS] 문장 A [SEP]로 표현한다.
- 문장 구별에는 두 방법을 함께 사용한다. [SEP]로 경계를 표시하고, 각 토큰에 문장 A/B를 나타내는 학습 가능한 세그먼트 임베딩을 더한다. 둘 중 하나를 선택하는 방식이 아니다.

c. ⇒ 주어진 토큰의 입력 표현 : 토큰 임베딩 + 세그먼트 임베딩 + 위치 임베딩



## 사전 학습

2가지 비지도 과제로 사전 학습

1. Masked LM : 입력 토큰의 일정 비율로 무작위로 가리고 가려진 토큰 예측

    a. 사용 이유: 목표 단어를 그대로 보여 주고 양방향으로 예측하면 자기 자신을 간접적으로 참조해 쉽게 정답을 복사할 수 있다. 그래서 예측 대상 토큰을 가리는 MLM을 사용해 좌우 문맥으로 복원하도록 학습한다.

    b. 방법

    - 마스크 토큰에 대응하는 최종 은닉 벡터를 어휘 전체에 대한 출력 소프트맥스에 입력
    - 전체 WordPiece 토큰 중 15%를 예측 대상으로 선택한다. 이 15% 모두가 [MASK]로 바뀌는 것은 아니다. 선택된 위치에서만 원래 토큰을 예측하고, 입력 전체를 복원하지 않는다.

    c. 단점: 일반적인 미세조정 입력에는 [MASK]가 없으므로 사전학습과 미세조정의 입력 분포가 달라진다.

    d. 완화 방법: 예측 대상으로 선택한 토큰을 항상 [MASK]로 바꾸지 않고, 무작위 토큰 대체와 원형 유지를 섞는다.

    - i번째 토큰 선택 → 80%확률로 MASK 토큰으로 바꿈, 10% 확률로 무작위 토큰으로 바꿈, 10%확률로 원래 토큰 그대로 → Ti를 사용해 원래 토큰 예측 → 교차 엔트로피 손실 계산

2. 다음 문장 예측(NSP): 단일 언어 말뭉치에서 쉽게 생성 가능한 이진 다음 문장 예측 과제

    a. 50%는 B를 A의 실제 다음 구간으로 골라 IsNext, 나머지 50%는 말뭉치의 무작위 구간으로 골라 NotNext를 붙인다. [CLS]의 최종 표현 C로 두 클래스를 분류한다.

    b. 사전학습 손실 = MLM 손실 + NSP 손실. 두 손실의 역전파로 BERT 내부 가중치를 함께 학습한다.

    c. → MLM·NSP로 사전학습한 BERT 가중치를 다운스트림 모델의 초기값으로 가져온다. 이후에는 분류·질의응답 등 실제 과제의 손실로 미세조정하며, 보통 NSP를 계속 수행하지 않는다.

## 미세조정

1. 기존의 흔한 방식: 두 텍스트를 각각 인코딩한 뒤 양방향 교차 어텐션으로 연결한다.

2. BERT: 두 텍스트를 하나의 입력으로 연결하고 자기어텐션으로 함께 인코딩한다. 따라서 두 문장 사이의 양방향 상호작용도 같은 과정에서 처리된다.

3. 과제 : 입력과 출력을 BERT에 연결, 모든 매개변수를 종단 간 방식으로 fine tuning

a. [입력]

- 바꿔쓰기 판별의 문장 쌍
- 함의 판별의 가설 전제 쌍
- 질의응답의 질문 지문 쌍
- 텍스트 분류·시퀀스 태깅: 문장 하나만 입력한다. 문장 B가 비어 있는 경우로 볼 수 있다.

b. [출력]

- 시퀀스 태깅: 각 토큰의 최종 표현 $T_i$를 출력층에 넣어 토큰별 레이블을 예측한다.
- 질의응답: 각 토큰의 표현 $T_i$로 정답 구간의 시작·끝 점수를 계산한다.
- 문장 분류: [CLS]의 최종 표현 C를 출력층에 넣어 함의·감성 등의 레이블을 예측한다.

c. → 사전학습에 비해 미세조정 비용이 적다. 논문에서는 같은 사전학습 모델에서 출발해 단일 Cloud TPU로 최대 약 1시간 안에 과제별 결과를 재현할 수 있다고 설명한다.

### Contribution

## 4. 실험 및 결과

### Dataset

1. 사전학습 데이터 : BooksCorpus의 8억 단어 & Wikipedia의 25억 단어

→ 문서 수준 말뭉치를 사용

2. GLUE : 다양한 자연어 이해 과제 모음

3. SQuAD v1.1 : 크라우드소싱으로 수집한 질문·답변 쌍 10만 개로 구성. 질문과 답이 포함된 Wikipedia 지문이 주어지면, 지문에서 정답 텍스트 구간을 예측하는 과제

4. SQuAD v2.0: 지문에 정답이 없는 질문까지 포함하여, 답 구간 추출과 정답 없음 판정을 함께 평가한다.

5. SWAG : 현실 상황에 기반한 상식 추론을 평가하는 문장쌍, 문장이 주어졌을 때 4지 선다로 다음 내용 고르는 문제

## Baseline

1. GLUE

    a. GLUE 적용: 문장 또는 문장 쌍 입력 → BERT가 문장 전체를 반영한 [CLS] 표현 생성 → 분류층이 정답 선택. 예측과 실제 정답 비교 → BERT와 분류층 함께 수정. 단, STS-B는 종류를 고르는 대신 두 문장의 유사도 점수를 예측.

2. SQuAD v1.1

    a. 질의응답 과제: 입력 질문과 지문을 하나의 시퀀스로 묶음

    b. SQuAD v1.1 적용: 질문과 지문을 함께 입력 → 각 토큰에 정답의 시작·끝 점수를 매김 → 가장 그럴듯한 연속 구간을 답으로 선택. 실제 정답 위치와 비교 → BERT와 시작·끝 점수를 매기는 가중치를 함께 수정. 답을 새로 쓰는 것이 아니라 지문에서 찾는 방식이며, 입력을 마스킹하지 않음.

    추가 데이터 활용: TriviaQA로 먼저 질의응답 미세조정을 하고, SQuAD v1.1로 다시 미세조정한다. 본문 설정은 학습률 5e-5, 배치 32, 3에포크다. 최종 성능은 아래 결과 항목에서 비교한다.

    c. 추가 데이터 활용: TriviaQA로 먼저 질의응답 미세조정을 하고, SQuAD v1.1로 다시 미세조정한다. 본문 설정은 학습률 5e-5, 배치 32, 3에포크다. 최종 성능은 아래 결과 항목에서 비교한다.

3. SQuAD v2.0

    a. SQuAD v2.0은 지문에 정답이 없는 질문도 포함한다. 정답이 없는 예시는 시작과 끝이 모두 [CLS]인 것으로 학습한다.

    b. 판정: 찾은 답의 점수와 “정답 없음” 점수를 비교 → 답이 있다고 충분히 확신할 때만 해당 구간 출력, 아니면 “정답 없음”. 얼마나 확신해야 하는지는 개발 세트로 정함. v1.1과 달리 TriviaQA는 사용하지 않음.

4. SWAG

    a. 주어진 문장과 후보 후속 문장을 연결한 입력을 선택지별로 4개 만든다. 각 입력의 [CLS] 표현에 동일한 학습 가능한 점수 벡터를 적용해 후보별 점수를 얻고, 네 점수에 소프트맥스를 적용해 가장 그럴듯한 후속 문장을 선택한다.

    b. 학습률 2e-5, 배치 크기 16으로 3에포크 동안 미세조정

5. 제거 실험

    a. 사전학습 과제의 영향

- **제거 실험의 목적: 사전학습 과제, 모델 크기, BERT의 활용 방식 중 어떤 요소가 성능에 영향을 주는지 분리해 비교한다.**
  **추가 비교 실험의 설정**
  No NSP: 양방향 MLM은 유지하고 NSP만 제거한다.
- BERT_BASE와 사전학습 데이터·미세조정 방식·하이퍼파라미터를 맞춰 비교한다.

b. LTR & No NSP: MLM 대신 왼쪽 문맥만 보는 정방향 언어 모델로 학습하고 NSP도 제거한다. 미세조정에서도 왼쪽 문맥만 보는 제약을 유지한다.

- 정방향 언어 모델로 학습
- +BiLSTM은 별도의 비교 설정: LTR & No NSP 위에 미세조정 시 무작위 초기화한 BiLSTM을 추가한다. 기본 LTR 모델에 원래 포함된 구성은 아니다.

c. 모델 크기 비교: 층 수·은닉 크기·헤드 수를 바꾸며 평가한다. 개발 세트 정확도는 미세조정 무작위 재시작 5회의 평균이다.

d. 특징 기반 접근법

- 특징 기반 방식의 장점: 과제별 구조가 필요한 경우 BERT 표현을 별도 모델의 입력으로 쓸 수 있다.
- 계산상의 장점: BERT의 토큰 표현을 한 번 미리 계산한 뒤, 고정된 표현 위에 더 저렴한 모델을 바꾸어 여러 실험을 할 수 있다.
- 미세 조정 방식 : BERT도 함께 학습
  - 예측이 틀리면 분류층과 bert 내부 가중치 함께 수정
- NER 특징 기반 방식: BERT가 각 단어의 문맥 표현을 만듦 → 별도 BiLSTM과 분류층이 사람·장소·조직 등의 이름표를 붙임. BERT는 고정하고 뒤의 모델만 학습. 대소문자와 문서 문맥을 활용하며, 단어가 여러 조각으로 나뉘면 첫 조각의 표현을 사용.

## 결과

1. GLUE

| System | MNLI-(m/mm) 392k | QQP 363k | QNLI 108k | SST-2 67k | CoLA 8.5k | STS-B 5.7k | MRPC 3.5k | RTE 2.5k | Average - |
|---|---|---|---|---|---|---|---|---|---|
| Pre-OpenAI SOTA | 80.6/80.1 | 66.1 | 82.3 | 93.2 | 35.0 | 81.0 | 86.0 | 61.7 | 74.0 |
| BiLSTM+ELMo+Attn | 76.4/76.1 | 64.8 | 79.8 | 90.4 | 36.0 | 73.3 | 84.9 | 56.8 | 71.0 |
| OpenAI GPT | 82.1/81.4 | 70.3 | 87.4 | 91.3 | 45.4 | 80.0 | 82.3 | 56.0 | 75.1 |
| $BERT_{BASE}$ | 84.6/83.4 | 71.2 | 90.5 | 93.5 | 52.1 | 85.8 | 88.9 | 66.4 | 79.6 |
| $BERT_{LARGE}$ | **86.7/85.9** | **72.1** | **92.7** | **94.9** | **60.5** | **86.5** | **89.3** | **70.1** | **82.1** |

- GLUE: BASE·LARGE 모두 표 1의 기존 비교 시스템을 능가했다. WNLI 제외 평균은 BASE 79.6, LARGE 82.1이다. 공식 GLUE 점수는 LARGE 80.5로 GPT의 72.8보다 7.7포인트 높다. 82.1과 80.5는 집계 범위가 다르다.
    - BERT_LARGE가 BERT_BASE보다 모든 GLUE 과제에서 우수했다. MNLI-m 정확도는 LARGE 86.7로 GPT의 82.1보다 4.6%포인트 높다.

2. SQuAD v1.1

| System | Dev | | Test | |
|---|---|---|---|---|
| | EM | F1 | EM | F1 |
| Top Leaderboard Systems (Dec 10th, 2018) | | | | |
| Human | - | - | 82.3 | 91.2 |
| #1 Ensemble - nlnet | - | - | 86.0 | 91.7 |
| #2 Ensemble - QANet | - | - | 84.5 | 90.5 |
| Published | | | | |
| BiDAF+ELMo (Single) | - | 85.6 | - | 85.8 |
| R.M. Reader (Ensemble) | 81.2 | 87.9 | 82.3 | 88.5 |
| Ours | | | | |
| $\text{BERT}_{\text{BASE}}$ (Single) | 80.8 | 88.5 | - | - |
| $\text{BERT}_{\text{LARGE}}$ (Single) | 84.1 | 90.9 | - | - |
| $\text{BERT}_{\text{LARGE}}$ (Ensemble) | 85.8 | 91.8 | - | - |
| $\text{BERT}_{\text{LARGE}}$ (Sgl.+TriviaQA) | **84.2** | **91.1** | **85.1** | **91.8** |
| $\text{BERT}_{\text{LARGE}}$ (Ens.+TriviaQA) | **86.2** | **92.2** | **87.4** | **93.2** |

SQuAD v1.1: TriviaQA를 거친 BERT_LARGE 단일 모델은 테스트 F1 91.8로 당시 최고 앙상블 nlnet의 91.7을 넘었다. BERT_LARGE 7개 앙상블+TriviaQA는 테스트 F1 93.2로 1.5포인트 개선했다. TriviaQA 없이도 개발 F1 하락은 0.1~0.4포인트였다.

3. SQuAD v2.0

| System | Dev | | Test | |
|---|---|---|---|---|
| | EM | F1 | EM | F1 |
| Top Leaderboard Systems (Dec 10th, 2018) | | | | |
| Human | 86.3 | 89.0 | 86.9 | 89.5 |
| #1 Single - MIR-MRC (F-Net) | - | - | 74.8 | 78.0 |
| #2 Single - nlnet | - | - | 74.2 | 77.1 |
| Published | | | | |
| unet (Ensemble) | - | - | 71.4 | 74.9 |
| SLQA+ (Single) | - | | 71.4 | 74.4 |
| Ours | | | | |
| $\text{BERT}_{\text{LARGE}}$ (Single) | 78.7 | 81.9 | 80.0 | 83.1 |

- SQuAD v2.0: BERT_LARGE 단일 모델 테스트 EM 80.0, F1 83.1. 이전 최고 시스템의 F1 78.0보다 5.1포인트 높다.

4. SWAG: BERT_LARGE 테스트 정확도 86.3%. GPT 78.0%보다 8.3%포인트, ESIM+ELMo 59.2%보다 27.1%포인트 높다.

| System | Dev | Test |
|---|---|---|
| ESIM+GloVe | 51.9 | 52.7 |
| ESIM+ELMo | 59.1 | 59.2 |
| OpenAI GPT | - | 78.0 |
| $\text{BERT}_{\text{BASE}}$ | 81.6 | - |
| $\text{BERT}_{\text{LARGE}}$ | **86.6** | **86.3** |
| Human (expert)† | - | 85.0 |
| Human (5 annotations)† | - | 88.0 |

5. 제거 실험 결과

a. nsp

| | Dev Set | | | | |
|---|---|---|---|---|---|
| Tasks | MNLI-m (Acc) | QNLI (Acc) | MRPC (Acc) | SST-2 (Acc) | SQuAD (F1) |
| $\text{BERT}_{\text{BASE}}$ | 84.4 | 88.4 | 86.7 | 92.7 | 88.5 |
| No NSP | 83.9 | 84.9 | 86.5 | 92.6 | 87.9 |
| LTR & No NSP | 82.1 | 84.3 | 77.5 | 92.1 | 77.8 |
| + BiLSTM | 82.1 | 84.1 | 75.7 | 91.6 | 84.9 |

- NSP 제거 효과: QNLI 88.4→84.9, MNLI-m 84.4→83.9, SQuAD F1 88.5→87.9로 하락했다. 양방향성 효과: No NSP와 LTR & No NSP를 비교하면 SQuAD F1이 87.9→77.8로 크게 낮아진다. +BiLSTM으로 84.9까지 회복해도 양방향 사전학습 모델에는 못 미치고, GLUE에서는 오히려 성능이 떨어졌다.

  b. 모델 크기: 충분히 사전학습하면 큰 모델이 작은 데이터셋에서도 유리하다. MRPC처럼 약 3,600개 학습 예시만 있어도 성능이 좋아졌다. 이는 단순히 다운스트림 데이터가 많아서 생긴 효과가 아니라 사전학습된 표현의 크기·표현력을 활용한 결과라는 것이 저자들의 해석이다.

| Hyperparams | | | | Dev Set Accuracy | | |
|---|---|---|---|---|---|---|
| #L | #H | #A | LM (ppl) | MNLI-m | MRPC | SST-2 |
| 3 | 768 | 12 | 5.84 | 77.9 | 79.8 | 88.4 |
| 6 | 768 | 3 | 5.24 | 80.6 | 82.2 | 90.7 |
| 6 | 768 | 12 | 4.68 | 81.9 | 84.8 | 91.3 |
| 12 | 768 | 12 | 3.99 | 84.4 | 86.7 | 92.9 |
| 12 | 1024 | 16 | 3.54 | 85.7 | 86.9 | 93.3 |
| 24 | 1024 | 16 | 3.23 | 86.6 | 87.8 | 93.7 |

c. 특징 기반 접근법

| System | Dev F1 | Test F1 |
|---|---|---|
| ELMo (Peters et al., 2018a) | 95.7 | 92.2 |
| CVT (Clark et al., 2018) | - | 92.6 |
| CSE (Akbik et al., 2018) | - | **93.1** |
| Fine-tuning approach | | |
| $\text{BERT}_{\text{LARGE}}$ | 96.6 | 92.8 |
| $\text{BERT}_{\text{BASE}}$ | 96.4 | 92.4 |
| Feature-based approach ($\text{BERT}_{\text{BASE}}$) | | |
| Embeddings | 91.0 | - |
| Second-to-Last Hidden | 95.6 | - |
| Last Hidden | 94.9 | - |
| Weighted Sum Last Four Hidden | 95.9 | - |
| Concat Last Four Hidden | 96.1 | - |
| Weighted Sum All 12 Layers | 95.5 | - |

- NER 특징 기반 비교에서는 마지막 4개 은닉층의 토큰 표현을 연결하는 방법이 가장 좋았다. 개발 F1 96.1로 BERT_BASE 전체 미세조정 96.4보다 0.3포인트 낮을 뿐이다. BERT_LARGE 미세조정은 개발 96.6, 테스트 92.8이었다. 특징 기반 96.1은 개발 점수이므로 테스트 점수와 직접 비교하지 않는다.

  → BERT는 전체를 미세조정해도, 고정된 특징 추출기로 사용해도 효과적이다. 다만 NER 테스트에서는 CSE 93.1이 BERT_LARGE 92.8보다 높으므로 이 실험까지 모두 최고 성능이라고 해석하면 안 된다.

# 5. 결론 (배운점)

## 의의

핵심 의의: MLM을 통해 모든 층에서 좌우 문맥을 함께 반영하는 심층 양방향 표현을 사전학습했다. 동일한 사전학습 BERT에 작은 출력층만 추가하고 전체를 미세조정하여, 문장 수준 과제와 토큰 수준 과제를 폭넓게 해결했다. 충분한 사전학습을 거치면 큰 모델이 학습 데이터가 적은 과제에서도 도움이 됨을 보였다.

## 한계

논문 설정에서 생각해 볼 한계(리뷰 해석): 사전학습 비용이 크고, 자기어텐션 비용은 시퀀스 길이의 제곱에 비례한다. 이 논문의 입력 길이는 최대 512토큰이다. [MASK]는 일반적인 미세조정 입력에 없으므로 사전학습과 미세조정의 불일치가 남는다. 또한 [CLS]를 미세조정 없이 곧바로 범용 문장 임베딩으로 해석해서는 안 된다.

## 배운 점

1. 양방향성은 좌우 문맥을 각각 학습한 뒤 연결하는 것과 다르다.

2. MLM·NSP는 사전학습 과제이고, 질의응답·분류·NER에서는 해당 과제의 손실로 미세 조정한다.
3. 출력층만 새로 추가한다는 말은 출력층만 학습한다는 뜻이 아니다.
4. 특징 기반 방식에서는 BERT를 고정하고 별도 모델만 학습한다.